國 立 成 功 大 學

電機工程研究所

碩 士 論 文

支付通道網路中提升效率的動態路由策略

Dynamic Routing for Enhanced Efficiency in Payment Channel Networks

研 究 生: 張嵩禾　Student: Sung-Ho Chang
指導教授: 斯國峰　Advisor: Kuo-Feng Ssu

Department of Electrical Engineering
National Cheng Kung University
Thesis for Master of Science
July 2025

中 華 民 國 一 百 一 十 四 年 七 月

國立成功大學

National Cheng Kung University

碩士論文合格證明書

Certificate of Approval for Master's Thesis

題目： 支付通道網路中提升效率的動態路由策略

Title： Dynamic Routing for Enhanced Efficiency in Payment Channel Networks

研究生：張嵩禾

本論文業經審查及口試合格特此證明

This is to certify that the Master's Thesis of SONG-HO CHANG

論文考試委員：

has passed the oral defense under the decision of the following committee members

高嘉宏　斯國峰

焦惠津

指導教授 Advisor(s)： 斯國峰

單位主管 Chair/Director： 江孟學

（單位主管是否簽章授權由各院、系（所、學位學程）自訂）

(The certificate must be signed by the committee members and advisor(s). Each department/graduate institute/degree program can determine whether the chair/director also needs to sign.)

2025/7/11

支付通道網路中提升效率的動態路由策略

張嵩禾

國立成功大學

電機工程研究所

摘要

區塊鏈技術的迅速發展，使傳統區塊鏈所面臨的擴展性問題愈發嚴峻，進而嚴重限制了其交易吞吐量（Transactions Per Second, TPS）。為了緩解此一瓶頸，支付通道網路作為第二層（Layer-2）擴展方案應運而生，旨在實現快速且低成本的鏈下交易，從而有效分擔主鏈的運算與儲存負載。然而，支付通道網路的部署與運作面臨諸多技術挑戰，其中「路由問題」被廣泛認為是實現高效支付的關鍵障礙。具體而言，支付通道網路中的路由需在一個高度動態、資金分佈不均且連線關係複雜的網路中，快速識別可行且高效的支付路徑。除此之外，如何在路由策略中權衡交易手續費與支付成功率，也是不可忽視的重要議題。

有效的路由機制至關重要，因為低效率的路由策略可能導致交易失敗、費用增加與用戶體驗下降，進而削弱支付通道網路作為擴展解決方案的實用價值與吸引力。因此，本研究聚焦於支付通道網路的路由問題的深入探討，系統性分析現有演算法的設計理念與性能瓶頸，並提出具體的改進策略，期望在提升交易成功率與降低成本之間取得良好平衡。最終，本研究致力於增進支付通道網路的整體效能與實務可行性，為區塊鏈擴展方案的廣泛應用奠定堅實基礎。

關鍵字：區塊鏈、鏈下、支付通道網路、路由

Dynamic Routing for Enhanced Efficiency in Payment Channel Networks

Sung-Ho Chang
Department of Electrical Engineering
National Cheng Kung University

The rapid advancement of blockchain technology has revealed a fundamental scaling problem that constrains transaction throughput (TPS) severely. To address the problem, Payment Channel Networks (PCNs) have emerged as a Layer-2 solution for facilitating fast and low-cost off-chain payments that reduce the main blockchain load. However, the PCN implementation presents several technical challenges. The implementation involves identifying optimal paths in complex and dynamically changing networks with uneven fund distribution. Furthermore, strategically balancing transaction fees and payment success rates during routing presents a significant challenge. Effective routing is crucial because inefficient paths can lead to failed transactions, increased costs, and a diminished user experience. This study thoroughly investigates the PCN routing problem by analyzing existing algorithms' strengths and weaknesses to propose targeted improvement strategies. Ultimately, this thesis aims to enhance the performance and practical utility of payment channel networks. This work also provides a way for broader adoption of scalable blockchain solutions.

Keywords—blockchain, off-chain, payment channel network (PCN), routing

Acknowledgments

First and foremost, I would like to express my sincerest gratitude to my advisor, Kuo-Feng Ssu. During my academic journey at National Cheng Kung University, he has always encouraged me and guided my research direction. I am immensely grateful for his unwavering efforts, which have enabled me to complete this thesis.

Thanks are also due to Professors Hewijin Christine Jiau and Chia-Hung Kao for their contributions to this research and for serving on my dissertation committee.

Thanks to the members of the Dependable Computing Laboratory, including Yu-Jung Chang, Tzi-Wei Ou, Fang-Yu Ko, Yu-Jen Yan, Yi-Hua Chang, You-Yao Chen, Yu-Wei Zheng, Yen-Wei Lin, Bo-Syun Ke, Xing-Ya Yang, Hao-Chen Liu, Gibson Teo, and Yin-Wei Chiu who gave me very helpful advice.

Finally, I would like to thank my family and friends for their support and company. Thanks for their precious time and encouragement.

Table of Contents

Chapter

List of Figures

List of Tables

Chapter 1

Introduction

Since the introduction with Bitcoin in 2009 [1], blockchain and cryptocurrency technologies have revolutionized financial systems that emerge as transformative network applications. In May 2025, Bitcoin's market capitalization exceeds $2.06 trillion USD [2]. In contrast to centralized financial systems, cryptocurrencies offer decentralization, transparent transaction traceability, and streamlined cross-border payments, so reliance on intermediaries can be reduced. However, the decentralized ledger of blockchain technology necessitates consensus among distributed nodes, which is achieved through broadcasting transactions across the network. This process incurs significant storage and communication overhead, so Bitcoin's throughput is limited to approximately 7 transactions per second (TPS) [3]. This scalability bottleneck demonstrates insufficiency for high-frequency retail in the payment systems such as Visa.

To overcome the limitations, Layer-2 scaling solutions have emerged to enable off-chain transactions while preserving blockchain's security guarantees. Payment Channel Networks (PCNs), such as Bitcoin's Lightning Network [4], Ethereum's Raiden Network [5], and Celer Network [6], are prominent examples. A PCN comprises inter-

connected payment channels. Each channel is established on-chain via a 2-of-2 multi-signature address [7] that requires two transacting nodes to deposit cryptocurrency to provide channel liquidity. These channels enable off-chain transactions. Payments are transferred directly in the channel, so on chain verification can be bypassed for better efficiency and lower fees. Within a PCN, multi-hop transactions allow funds to traverse multiple channels to reach a recipient, even in the absence of a direct channel. Hash Time Locked Contracts (HTLCs) [8], which combine cryptographic hashes with timelocks, ensure security in these multi-hop transfers by preventing intermediaries from withholding funds. Upon channel closure, its final state is settled on-chain, and the funds are released. The off-chain approach supports scalable and low-cost cryptocurrency payments, so the PCN has become a promising solution for everyday transactions.

One of the primary challenges for current PCNs is designing efficient transaction routing schemes. When multiple transactions occur concurrently in the network, the system must identify feasible paths for each pair of sender and receiver nodes. Furthermore, it must ensure that all channels along the selected payment path possess sufficient balance to complete the transaction. In fact, within PCNs, the balances of individual payment channels are highly dynamic. Each new transaction updates the remaining amount within the affected channels. A chosen path for a transaction may fail due to insufficient current channel balance.

This thesis introduces a novel two-phase routing algorithm for improving transaction efficiency, success rates, and cost-effectiveness within PCNs. The algorithm leverages either local or global network information to select an initial payment path, based on the estimated success probability and transaction fees.

Unlike existing methods that probe paths sequentially, this algorithm concurrently probes alternative paths during the initial attempt. Should the initial payment fail, the algorithm will use the probing results to select an optimal path for retransmitting the remaining amount with sufficient channel capacity and lower fees. This two-phase strategy improves latency, success rates, and costs.

The thesis describes an approach to enabling scalable and reliable cryptocurrency payments. With initial routing and failure recovery, the algorithm tackles real-world challenges, such as high transaction volumes in retail or cross-border payments, where low latency and high reliability are critical.

The main contributions of this thesis are:

1. **A novel two-phase routing algorithm** that integrates initial path selection with parallel probing of alternative paths provides real-time feedback for efficient retries.

2. **A path scoring mechanism**, which evaluates initial paths based on both success probability and transaction fees.

3. **A smart retry mechanism** utilizes probing results to select optimal paths for retransmission for failed payments.

4. **An empirical validation framework** that proposes evaluation through simulations on realistic PCN topologies to demonstrate the performance of the algorithm.

By combining efficient initial routing with intelligent failure recovery, this algorithm can enhance PCN performance and provide the way for scalable cryptocurrency transactions in complex and dynamic environments.

Chapter 2

Related Work

Routing within PCNs presents a significant research challenge, primarily due to limited channel capacity and highly dynamic network conditions. Most current routing algorithms necessitate that the initiating node possesses global network topology information to compute end-to-end payment paths from sender to receiver. These methods typically rely on path probing to assess residual channel capacities before transaction execution.

For instance, Deter-Pay [9] employs a greedy strategy and probes to select feasible paths with the lowest fees. Similarly, CoinExpress [10] and Flash [11] also leverage probing-based approaches. Flash segments large payments into multiple sub-payments and probes channel states to acquire necessary information prior to execution. EPA-Route [12] further proposes a routing and probing mechanism that simultaneously balances high success rates with low transaction fees. EPA-Route uses local routing tables at each node to identify potential paths. It then filters and prunes these paths based on hop feasibility. After path discovery, EPA-Route aggregates multiple valid paths. It segments payments into sub-payments and distributes them across these paths

to enhance overall efficiency. However, this "probe-then-pay" approach inherently introduces substantial transaction delays, as routing decisions are contingent upon the completion of the probing process.

Beyond probe-based methods, several studies [13–15] concentrate on enhancing routing efficiency and channel longevity through dynamic fee adjustments. The core idea is to increase fees for unbalanced or heavily utilized channels while lowering fees for balanced channels. This guides traffic away from potential bottleneck paths. Fence [13] exemplifies this approach by setting strategic fee rates. This prevents rapid fund depletion on one side of a channel due to asymmetric payment flows. PCN sustainability and long-term throughput are enhanced. MPCN-RP [14] further incorporates diverse transaction costs. The system integrates channel imbalance and hop fees into its routing model to identify paths optimized for both total cost and path length. Despite the mechanisms for guiding traffic and balancing funds, the implementation encounters challenges. Frequent fee fluctuations introduce uncertainty in transaction costs. This is particularly detrimental to high-frequency or time-sensitive applications. Additionally, malicious nodes could potentially manipulate fees to disrupt normal routing, and then security risks are introduced.

Current literature primarily concentrates on enhancing initial payment success rates. Methods, such as AutoTune [16], PACO [17], and VEIN [18], aim to enhance path selection efficiency and resource utilization. Despite advancements in routing design, the systematic exploration of retransmission strategies following payment failures remains limited. This limitation is particularly critical given the constraints of limited channel capacity.

Existing research has made notable strides in enhancing initial transaction success rates and fee adjustment strategies. The reliance on real-time channel state probing introduces substantial delays. Furthermore, the recovery and retransmission strategies for payment failures remain a critical issue. This study aims to develop low-latency routing mechanisms and robust retransmission protocols for enhancing the scalability and reliability of PCNs.

Chapter 3

System Model

This chapter introduces the system model of the PCN, including its structure, routing mechanisms, and related assumptions.

3.1 System Overview

The proposed system aims to significantly enhance the overall transaction success rate, reduce transaction fees, and minimize transaction latency. Transaction latency is specifically defined as the duration from the initiation of a transaction request until either the complete delivery of funds to the recipient or the system's confirmation of failure.

Upon receiving a transaction request, the system employs a routing algorithm to identify optimal payment pathways. A transaction is considered successful only when the entire requested amount reaches the recipient. This algorithm is designed to increase the success rate while carefully balancing transaction cost and latency, ensuring transactions complete with reduced fees without compromising reliability or incurring substantial delays.

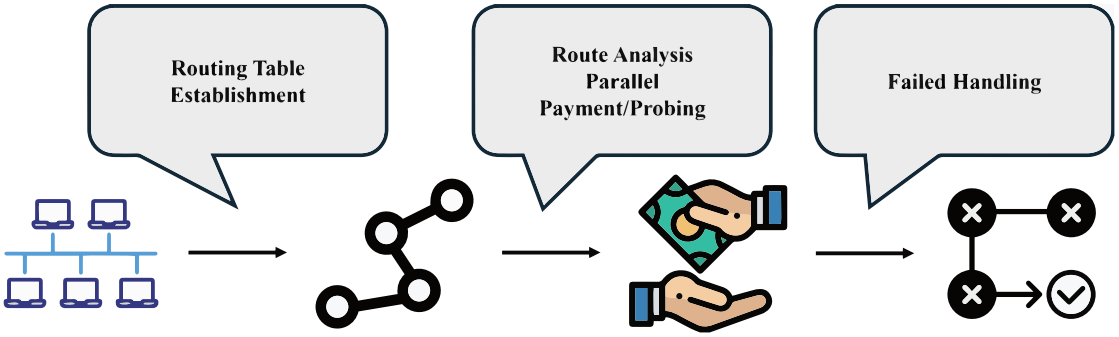

Figure 3.1: System overview.

Figure 3.1 depicts an overview of the system. The routing algorithm comprises three primary components: Routing Table Establishment, Route Analysis with Parallel Payment Probing, and Failure Handling. Each component will be elaborated upon in subsequent sections.

3.2 System Assumption

The PCN serves as a Layer-2 solution designed to enhance the scalability of blockchain transactions and reduce latency. The core concept involves establishing payment channels that enable two parties to perform multiple off-chain transactions without submitting each transaction to the underlying blockchain. Individual channels are interconnected to form a network. This network enables efficient value transfer across multiple nodes. To incentivize nodes to participate in routing services, most protocols implement a routing fee mechanism, which allows intermediary nodes to earn a reward for forwarding transactions. This section will introduce the concepts of payment channels, the structure of the PCN, and the design of routing fees in subsequent subsections.

Payment Channels: A payment channel is jointly established by two participants through the deployment of a smart contract. Both parties deposit funds into a 2-of-2 multi-signature wallet that serves as the basis of the channel. The opening and closing

of the channel are recorded on the underlying blockchain, and all transactions during the channel's lifetime are conducted off-chain. This multi-signature wallet facilitates a bidirectional payment channel. Each participant holds a certain amount of available balance. In each transaction, one party's balance decreases while the other party's balance increases by the corresponding amount.

As depicted in Figure 3.2a, Alice and Bob deposit funds into the payment channel. Subsequently, Alice sends 1 satoshi to Bob, as illustrated in Figure 3.2b. Following this transaction, as shown in Figure 3.2c, Alice's balance decreases by 1 satoshi, and Bob's balance increases by 1 satoshi, so completing an off-chain transfer of value is completed.

Payment Channel Network: A payment channel network comprises multiple payment channels and participating users. In instances where no direct payment channel exists between a sender and a receiver, funds can be routed through intermediate nodes. As depicted in Figure 3.3, nodes within the network can forward funds to other nodes via connected intermediate nodes and their corresponding payment channels.

This study assumes that the network topology of the PCN is periodically disseminated and updated among nodes using a gossip protocol. Each node can access information, such as the overall network topology, the fee models of other nodes, and the total capacity of payment channels. However, the real-time channel balances are considered private and cannot be directly obtained.

Transaction Fee: In a PCN, the transaction fee typically comprises two components: a *base fee* and a *proportional fee*. The total transaction fee model can be expressed as follows.

Let each path q be defined as a sequence of adjacent payment channels (i, j). For each channel (i, j), the following are defined:

- $\beta_{i,j}$: the *base fee* charged by channel (i, j);
- $\gamma_{i,j}$: the *proportional fee rate* set by channel (i, j).

Then, the total fee incurred by sending an amount $z_{\rho,q}$ along path q is given by:

$$C(z_{\rho,q}, q) = \sum_{(i,j)\in q} (\beta_{i,j} + \gamma_{i,j} \cdot z_{\rho,q}) \tag{1}$$

This model encapsulates the fact that the total transaction fee for a sub-transaction is contingent upon the number of hops along the selected path, the fee configuration of each channel, and the amount being transferred. The model is widely adopted in Layer-2 payment protocols, including the Lightning Network.

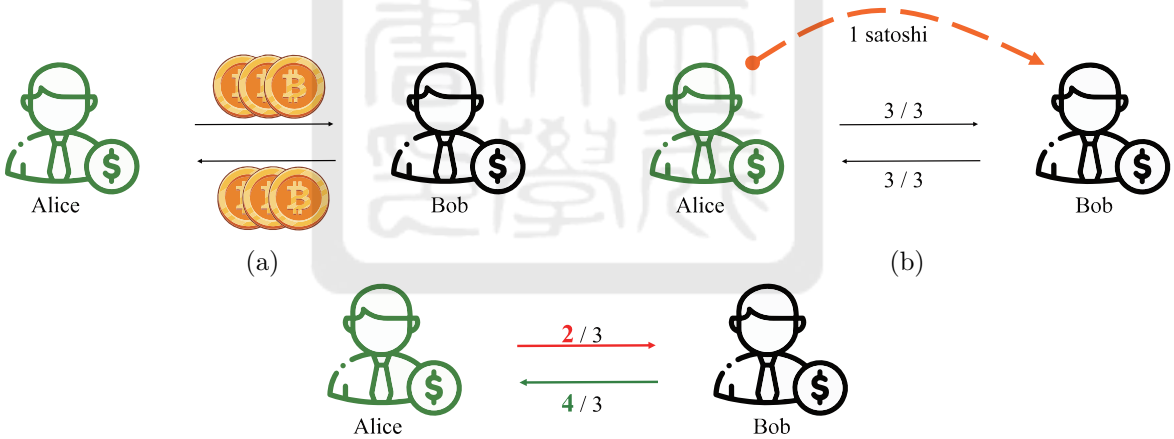

Figure 3.2: Example of the payment channel.

3.3 Problem Formulation

This study investigates a PCN environment where multiple transactions execute concurrently within a fixed time interval T. Each node within the network may process

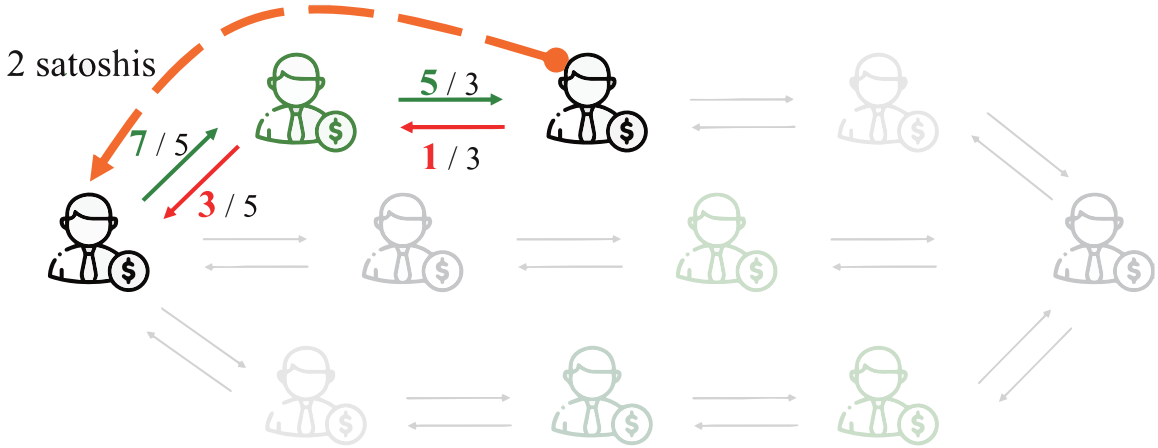

Figure 3.3: Payment channel network.

multiple incoming fund transfer requests in parallel. A key operational characteristic is that intermediate channels involved in a transaction remain locked until the fund transfer is finalized. Given the dynamic and unpredictable nature of individual channel balances, it is challenging to ascertain whether a path possesses sufficient liquidity using only predefined or static information. To address this challenge, a routing algorithm is proposed to optimize path selection and transaction amount allocation within such a dynamic environment.

Let $\mathcal{R}$ denote the set of all transaction requests generated during the interval T. Each transaction request $\rho \in \mathcal{R}$ is represented as a tuple $\rho = (u_\rho, v_\rho, a_\rho, t_\rho)$, where u_ρ and v_ρ denote the source and destination nodes, respectively. a_ρ is the transaction amount and t_ρ is the initiation time.

For each transaction ρ, a set of candidate paths $\mathcal{Q}^\rho$ connecting u_ρ to v_ρ is assumed. Each path $q \in \mathcal{Q}^\rho$ is defined as $q = (u_\rho, v_1^q, v_2^q, \ldots, v_l^q, v_\rho)$, where $v_1^q, v_2^q, \ldots, v_l^q$ are intermediate nodes along the path.

The proposed routing algorithm supports multipath payments. A single transaction is segmented into multiple sub-transactions routed through distinct paths. Let $z_{\rho,q}$ denote the sub-amount of transaction ρ assigned to path q. A binary variable $\delta_\rho \in \{0, 1\}$

is defined to indicate whether transaction ρ is successfully completed ($\delta_\rho = 1$) or not ($\delta_\rho = 0$).

The optimization problem is formulated with the following objectives:

1. **Maximize the total amount of successfully completed transactions.** As delineated in Equation 3.1, this objective seeks to prioritize the successful completion of transactions for increasing the total transferred amounts.

$$\max \sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho} z_{\rho,q} \cdot \delta_\rho \tag{3.1}$$

2. **Minimize the total transaction fees.** As presented in Equation 3.2, this objective aims to reduce the total cost incurred from routing payments. $C(z_{\rho,q}, q)$ denotes the fee for sending amount $z_{\rho,q}$ along path q.

$$\min \sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho} C(z_{\rho,q}, q) \tag{3.2}$$

Constraints

The optimization is subject to the following constraints:

1. **Transaction fulfillment constraint.** As expressed in Equation 3.3, the total allocated sub-amounts for each transaction must meet or exceed its required amount.

$$\sum_{q \in \mathcal{Q}^\rho} z_{\rho,q} \geq a_\rho \cdot \delta_\rho, \quad \forall \rho \in \mathcal{R} \tag{3.3}$$

2. **Channel capacity constraint.** For each channel (i, j) with available capacity $cap_{i,j}$, the aggregate flow through the channel from all transactions must not exceed this capacity. The variable $z_{\rho,q}$ is included in the sum if $(i, j) \in q$.

$$\sum_{\rho \in \mathcal{R}} \sum_{q \in \mathcal{Q}^\rho : (i,j) \in q} z_{\rho,q} \leq cap_{i,j}, \quad \forall (i, j) \in E \tag{3.4}$$

3. **Non-negativity constraint.** All allocated sub-transaction amounts must be non-negative.

$$z_{\rho,q} \geq 0, \quad \forall \rho \in \mathcal{R},\ q \in \mathcal{Q}^{\rho} \tag{3.5}$$

4. **Binary success indicator.** The transaction success variable δ_{ρ} is constrained to be binary.

$$\delta_{\rho} \in \{0, 1\}, \quad \forall \rho \in \mathcal{R} \tag{3.6}$$

Chapter 4

Proposed Solution

This chapter provides a detailed explanation of the routing algorithm designed for PCN. The algorithm consists of three components: Routing Table Establishment, Route Analysis and Parallel Payment and Probing, and Failed Handling. The complete system architecture is illustrated in Figure 4.1.

4.1 Notation and Definitions

Table 4.1 lists the primary symbols used throughout the proposed routing algorithm. These notations are essential for the detailed operations in each phase.

4.2 Routing Table Establishment

This study assumes that each node within the Payment Channel Network (PCN) can acquire the overall network topology and initial channel capacities via the gossip protocol. In the initial phase of the system, a routing table is constructed for each node by executing the candidate path computation process defined in Algorithm 1.

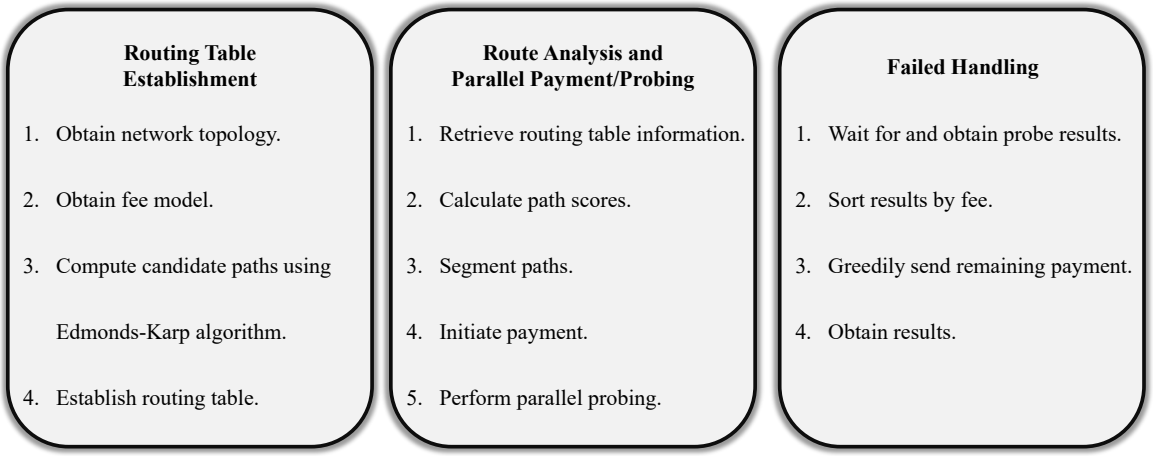

Figure 4.1: Flow chart of the system.

TABLE 4.1: List of Notations

Symbol	**Description**
p	A specific path in the payment network
p_i	An individual path (often indexed, e.g., $p_1, p_2, \ldots$)
$Y(p)$	Capacity of path p
A	Total transaction amount to be split
A_i	Amount of the transaction allocated to path p_i
k	Maximum number of paths to use for splitting, or number of top paths to probe
$\mathcal{P}$	Set of all candidate paths for the transaction
$\mathcal{S}$	Subset of currently selected paths from $\mathcal{P}$
TotalFee(p)	Total forwarding cost for path p
$\alpha_{[v_i,v_j]}$	Base fee for channel (v_i, v_j)
$\beta_{[v_i,v_j]}$	Fee rate for channel (v_i, v_j)
G	The current state of the network graph
A_{rem}	Remaining transaction amount that needs to be paid
$\mathcal{R}$	Set of successful probing results, each containing a path p_i and its confirmed capacity $Y(p_i)$
γ	Splitting threshold

Algorithm 1: Candidate Path Computation from Node v_s

Input:
Payment channel graph $G = (V, E)$;
Channel capacities $c_{[v_i,v_j]}$, fees $\alpha_{[v_i,v_j]}$, $\beta_{[v_i,v_j]}$;
Source node v_s.
Output:
Edge-disjoint candidate paths $\{\mathcal{P}_{v_s \to v_d} \mid v_d \in V, v_d \neq v_s\}$.
Definition:
$\mathcal{P}_{v_s \to v_d}$: Set of edge-disjoint candidate paths from v_s to v_d.
G^{res}: Residual graph with updated capacities after each path discovery.
p: A path from v_s to v_d, found via BFS.
TotalFee$(p) = \sum_{(v_i,v_j)\in p} \alpha_{[v_i,v_j]} + \beta_{[v_i,v_j]} \cdot A$: Forwarding cost of p for amount A.

foreach $v_d \in V, v_d \neq v_s$ **do**
  Initialize residual graph G^{res} with capacities $c_{[v_i,v_j]}$
  $\mathcal{P}_{v_s \to v_d} \leftarrow \emptyset$
  while *a path p exists from v_s to v_d in G^{res} via BFS* **do**
    Let $p = \{v_s, \ldots, v_d\}$
    $Y(p) \leftarrow \min\{c_{[v_i,v_j]} \mid (v_i, v_j) \in p\}$
    if $Y(p) > 0$ **then**
      $A \leftarrow Y(p)$
      $\alpha(p) \leftarrow \sum_{(v_i,v_j)\in p} \alpha_{[v_i,v_j]}$
      $\beta(p) \leftarrow \sum_{(v_i,v_j)\in p} \beta_{[v_i,v_j]}$
      Add $(p, A, \alpha(p), \beta(p))$ to $\mathcal{P}_{v_s \to v_d}$
      foreach $(v_i, v_j) \in p$ **do**
        $c_{[v_i,v_j]} \leftarrow c_{[v_i,v_j]} - A$
        $c_{[v_j,v_i]} \leftarrow c_{[v_j,v_i]} + A$
    else
      Identify bottleneck edge (v_b, v_b') on p with minimal capacity
      Set $c_{[v_b,v_b']} \leftarrow 0$ in G^{res}
    Remove all edges (v_i, v_j) from G^{res} where $c_{[v_i,v_j]} \leq 0$
return $\{\mathcal{P}_{v_s \to v_d} \mid v_d \in V, v_d \neq v_s\}$

To discover multiple candidate paths to each destination node v_d (where $v_d \neq v_s$), this study proposes a breadth-first search (BFS)-based edge-disjoint path computation algorithm. The algorithm first initializes a residual graph based on the original channel capacities $c_{[v_i,v_j]}$, and then repeatedly performs BFS to identify feasible paths from the source node v_s to v_d.

Once a path p is identified, the algorithm calculates the maximum transferable amount $Y(p)$ along the path, which is defined as the minimum residual capacity among all edges on the path. If $Y(p) > 0$, the algorithm further computes the total base fee and total fee rate along the path, and then stores the path along with its attributes as a candidate path.

The residual graph is subsequently updated based on $Y(p)$. The capacity of each forward edge is reduced by $Y(p)$, while the capacity of the corresponding reverse edge is increased by $Y(p)$ to support potential backward transmissions. If $Y(p) = 0$, the algorithm identifies the bottleneck edge and removes it from the residual graph to facilitate the discovery of alternative paths.

4.3 Phase I: Routing Analysis with Concurrent Payment and Probing

4.3.1 Path Scoring

The algorithm proposes a composite scoring function to prioritize candidate paths for a given transaction request. Each path p is evaluated based on three criteria: channel capacity, transaction fee, and channel usage frequency. The scoring function is defined as follows:

$$\text{Score}(p) = \alpha \cdot \widetilde{Y(p)} - \beta \cdot \widetilde{F(p)} - \gamma \cdot \widetilde{U(p)} \tag{4.1}$$

where:

- $\widetilde{Y(p)}$ is the normalized maximum payable amount along path p (i.e., normalized $Y(p)$);
- $\widetilde{F(p)}$ is the normalized total forwarding fee to send amount A along p, computed as $\sum_{(v_i,v_j)\in p} \left(\alpha_{[v_i,v_j]} + \beta_{[v_i,v_j]} \cdot A\right)$;
- $\widetilde{U(p)}$ is the normalized maximum usage frequency among all channels on path p, defined as $\max_{(v_i,v_j)\in p} U_{[v_i,v_j]}$;
- α, β, γ are user-defined weighting parameters that control the trade-off between capacity, fee, and congestion.

Each candidate path is assigned a score utilizing Equation 4.1. All paths are subsequently sorted in descending order of their scores. The top-ranked paths are preferred for subsequent payment and probing procedures.

4.3.2 Channel Usage Frequency Update Mechanism

To quantify the usage frequency of each channel, as referenced in the path scoring model, each node within the system locally maintains historical records of its owned channels. Whenever a channel (u, v) is utilized for a transaction, the node updates its usage frequency based on the elapsed time since its last usage. Specifically, an *Exponentially Weighted Moving Average (EWMA)* is applied to emphasize recent activity and to adapt to temporal usage dynamics.

This mechanism enables the system to reflect the real-time activeness of each channel and to incorporate such information into routing decisions. The frequency update procedure is formally described in Algorithm 2.

Algorithm 2: Update Usage Frequency of Channel $U_{[v_i,v_j]}$

Input:
Channel (v_i, v_j);
EWMA update parameter $\lambda \in (0, 1)$
Output:
Updated $U_{[v_i,v_j]}$ and last-used timestamp
Definition:
$U_{[v_i,v_j]}$: Estimated usage frequency of channel (v_i, v_j) based on Exponentially Weighted Moving Average (EWMA).
$last\_used_{[v_i,v_j]}$: The last time channel (v_i, v_j) was used.
λ: Smoothing factor for EWMA, balancing recent usage and historical average.

```
now ← current system time
if channel (v_i, v_j) was previously used then
    Δt ← now − last_used_[v_i,v_j]
    if Δt > 0 then
        U_[v_i,v_j] ← λ · 1/Δt + (1 − λ) · U_[v_i,v_j]
else
    U_[v_i,v_j] ← 1                          // Initialize on first use
last_used_[v_i,v_j] ← now
```

4.3.3 Trusted Nodes for Real-Time Channel State Inquiry

To enhance the timeliness and accuracy of channel state information, the proposed system introduces the concept of *trusted nodes*. A trusted node refers to a counterparty that has exhibited a frequent transaction history with the sender. In practice, the trusted nodes may be individuals or merchants with whom the user has established trust, such as friends, family, or business partners.

The system offers an API interface that enables the sender to query trusted nodes in real-time for up-to-date information regarding the residual capacity and usage frequency of specific payment channels. This mechanism complements the static topology disseminated through the gossip protocol. Routing decisions can be made by the more updated information.

Algorithm 3: Adaptive Multi-Path Payment Splitting

Input:
Candidate paths $\mathcal{P} = \{p_1, p_2, \dots\}$ with capacities $Y(p)$;
Transaction amount A;
Minimum success rate threshold θ;
Maximum path count k
Output:
Split amounts $\{A_i \mid p_i \in \mathcal{S}\}$
Definition:
θ: Minimum success rate threshold for path selection
$\mathcal{S}$: Subset of selected paths from $\mathcal{P}$
T: Total capacity of the currently selected paths in $\mathcal{S}$
ρ: Minimum remaining capacity ratio among selected paths

Initialize $\mathcal{S} \leftarrow \{p_1\}$
while *True* **do**
  $T \leftarrow \sum_{p_i \in \mathcal{S}} Y(p_i)$
  if $T = 0$ **then**
    raise error: insufficient capacity
  foreach $p_i \in \mathcal{S}$ **do**
    $A_i \leftarrow A \cdot \frac{Y(p_i)}{T}$
  $\rho \leftarrow \min_{p_i \in \mathcal{S}} \left(\frac{Y(p_i) - A_i}{Y(p_i)} \right)$
  if $\rho \geq \theta$ ***or*** $|\mathcal{S}| = |\mathcal{P}|$ ***or*** $|\mathcal{S}| = k$ **then**
    break
  Add next path p_j to $\mathcal{S}$
return $\{A_i \mid p_i \in \mathcal{S}\}$

By leveraging trusted nodes, the sender can enhance the quality of path selection and traffic distribution. Payment success rates and overall routing efficiency can be improved.

4.3.4 Adaptive Multi-Path Payment Splitting Mechanism

An adaptive multi-path splitting mechanism is designed to efficiently allocate payment amounts across multiple paths, as presented in Algorithm 3. The algorithm dynamically assigns portions of the total payment to candidate paths based on their maximum transferable capacities. The algorithm also ensures that the minimum success rate among the selected paths does not fall below the system-defined threshold θ.

4.3.5 Path Probing and Payment Execution Procedure

Algorithm 4 outlines the procedure for executing payment transfers and initiating path probing. The system first attempts to transmit the segmented payment amounts through the initially selected paths.

Subsequently, the system evaluates the remaining unutilized candidate paths, ranks them based on their total forwarding fees that comprise both fixed and proportional components. The top-k paths with the lowest forwarding fees are selected and issued as probing tasks to collect up-to-date information on channel availability.

Finally, if the payment is successfully completed via the initially selected paths, the task will be designated as successful and the total incurred fees will be calculated. Conversely, if the available capacity is insufficient, the task will be designated as failed and the subsequent retry mechanism for failed payments will be proceeded.

4.4 Phase II: Failed Handling

Algorithm 5 introduces the fallback mechanism that is invoked after the failure of the initial routing attempt. The system utilizes previously collected probing results to identify alternative paths with sufficient capacity. For each probed path, a threshold is applied to the probed capacity to reduce the likelihood of failure during reattempts. Based on this threshold-adjusted capacity, the system allocates the remaining unpaid amount across eligible fallback paths.

Each selected fallback path subsequently executes the payment. If the entire remaining amount is successfully transferred via these paths, the payment task will be designated as success. If only a portion of the amount is transferred, the entire transaction will be defined as failure.

Algorithm 4: Path Probing and Payment Execution

Input:
Candidate paths $\{(p, Y(p), \alpha(p), \beta(p))\}$ sorted by preference;
Assigned split amounts $\{A_i\}$ for selected paths
Output:
Payment status (success/failure), total fee
Definition:
$\alpha(p)$: Base fee for path p (sum of base fees of channels in p)
$\beta(p)$: Fee rate for path p (sum of fee rates of channels in p)
$\mathcal{C}$: Set of remaining unselected candidate paths
τ: Timeout parameter for probing task
ProbeTask(p, τ): A task to probe path p with timeout τ

`// Step 1: Execute payment on selected paths`
foreach (p_i, A_i) *in selected paths* **do**
    Attempt to transfer A_i through p_i

`// Step 2: Probe additional candidate paths by fee`
Let $\mathcal{C} \leftarrow$ remaining unselected candidate paths
Sort $\mathcal{C}$ by total forwarding cost:

$$\texttt{// TotalFee}(p) = \sum_{(v_i, v_j) \in p} \alpha_{[v_i, v_j]} + \beta_{[v_i, v_j]} \cdot A$$

Let $\mathcal{C}_{top} \leftarrow$ top k paths with lowest total fee in $\mathcal{C}$
foreach $p \in \mathcal{C}_{top}$ **do**
    Create and enqueue probing task ProbeTask(p, τ)

`// Step 3: Finalize payment result`
if *channel capacity is sufficient* **then**
    Mark task as successful; compute total fee
else
    Mark task as failed due to insufficient capacity
return *Payment status and total fee*

Algorithm 5: Fallback Routing Using Probed Paths

Input:
Remaining amount A_{rem};
Probing results $\mathcal{R} = \{(p_i, Y(p_i))\}$;
Splitting threshold γ;
Network graph G and current payment task
Output:
Updated payment task (success/failure), total fee
Definition:
Current payment task: The overall payment operation being processed
$\mathcal{F}$: List of fallback paths chosen for payment, each with an assigned amount (p, A_s)
A_c: Current remaining capacity/amount to fulfill from A_{rem}
A_s: Amount to be sent through a specific fallback path p
success: Boolean variable indicating the overall success of fallback routing
fee_{total}: Accumulative total fee incurred for successful transfers

Initialize $\mathcal{F} \leftarrow [\,]$
$A_c \leftarrow A_{rem}$ `// Remaining amount to fulfill`
foreach $(p, Y(p)) \in \mathcal{R}$ **do**
  if $A_c \leq 0$ **then**
    break `// Target amount fulfilled`
  $A_s \leftarrow \min(A_c,\ Y(p) \cdot (1 - \gamma))$ `// Apply success margin`
  if $A_s > 0$ **then**
    Append (p, A_s) to $\mathcal{F}$
    $A_c \leftarrow A_c - A_s$
if $\mathcal{F}$ *is empty* **then**
  Mark payment task as **failed**; no feasible fallback paths
else
  success $\leftarrow$ True, $fee_{total} \leftarrow 0$
  foreach $(p, A_s) \in \mathcal{F}$ **do**
    if *transfer of* A_s *through* p *is successful* **then**
      Update network state; fee_{total} += TotalFee(p)
    else
      success $\leftarrow$ False; **break** `// Transfer failed, stop further attempts`
  if *success* ***and*** $A_c \leq 0$ **then**
    Mark payment task as **successful**; return $fee \leftarrow fee_{total}$
  else if *success* ***and*** $A_c > 0$ **then**
    Mark payment as **partially successful**; update $A_{rem} \leftarrow A_c$
  else
    Mark payment task as **failed**; preserve A_{rem}

Chapter 5

Performance Evaluation

The evaluation of the system was performed on Python 3.12.3 running on an AMD Ryzen 5 3600 CPU. A network comprising 100 nodes was generated using the Barabasi-Albert model [19], with the average node degree configured to 5. The channel capacity for each channel was set using the data from bitcoinvisuals.com [20]. Channel transaction fees were generated based on the average fee values and median values provided by 1ml.com [21]. For transaction amounts, this study sampled credit card transaction amounts from the Credit Card Fraud Detection dataset [22]. Transaction partner combinations were generated with the research data from Flash [11]. This approach aimed to simulate realistic transaction patterns. Regarding the simulator setup, this system employed Poisson distribution simulations to randomly generate transaction arrival times for various transaction frequencies (TPS). The single-hop latency in the experiment was set to 30 ms, referencing Deter-Pay [9]. Additionally, transactions were enabled to execute concurrently.

TABLE 5.1: Simulation Parameters

Parameter	Value
Number of Nodes	100
Number of Channels	521
Average Channel Capacity	8,929,583 satoshis
Median Channel Capacity	1,589,612 satoshis
Transaction Frequency (TPS)	10—100
Transaction Amount Range	0.0—25,691.16 USD
Simulation Duration	30—60 s
Single-hop Latency	30 ms

5.1 Comparison Algorithms

1. **Shortest Path**: Shortest path routing is currently employed in the Lightning Network to determine the path between senders and receivers for payments.

2. **Flash** [11]: Flash differentiates between high-value "elephant" payments and low-value "mice" payments. The strategy can improve success rates.

3. **Deter-Pay** [9]: Deter-Pay combines precomputed candidate paths with a probing-based balance reservation mechanism.

5.2 Performance Under Different Workloads

The experiment aims to verify the algorithm's comprehensive performance under varying transaction frequencies. The experimental setup configures the transaction frequency per second from 10 TPS to 100 TPS. The transaction success rate, execution time, and transaction fees incurred per satoshi are recorded with the varying frequencies.

Transaction Success Rate: Figure 5.1 illustrates the variation in transaction success rate as the workload (TPS) increases. The proposed method consistently outperforms other protocols, including Deter-Pay, Flash, and Shortest Path, with all tested work-

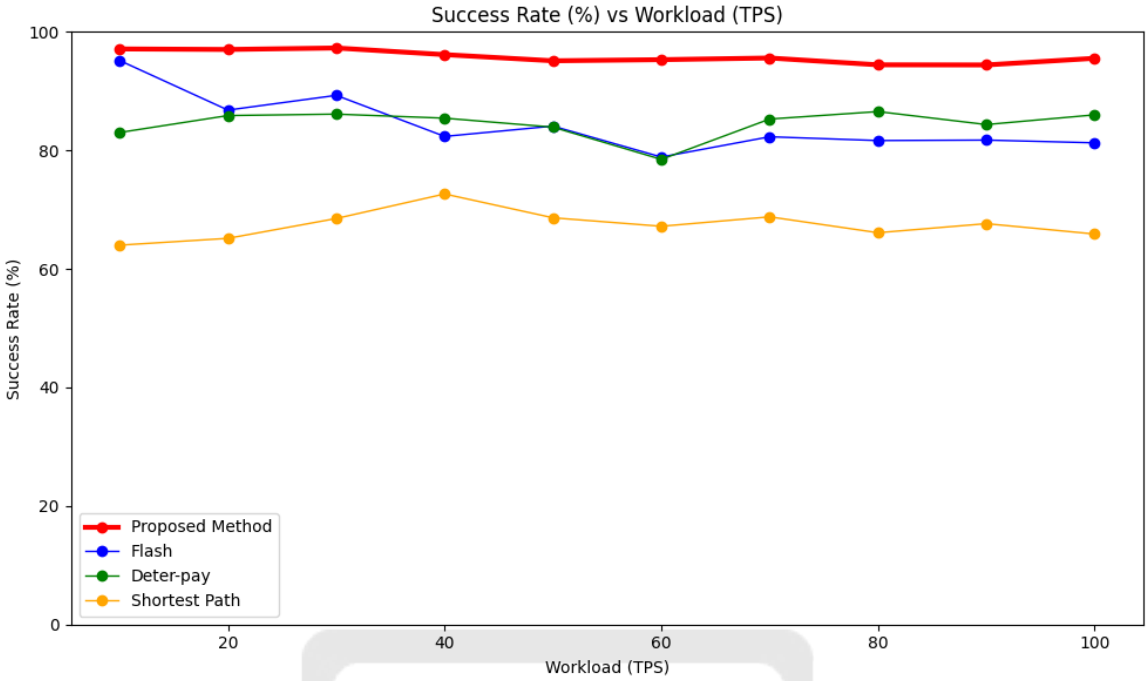

Figure 5.1: Success rate under different workloads.

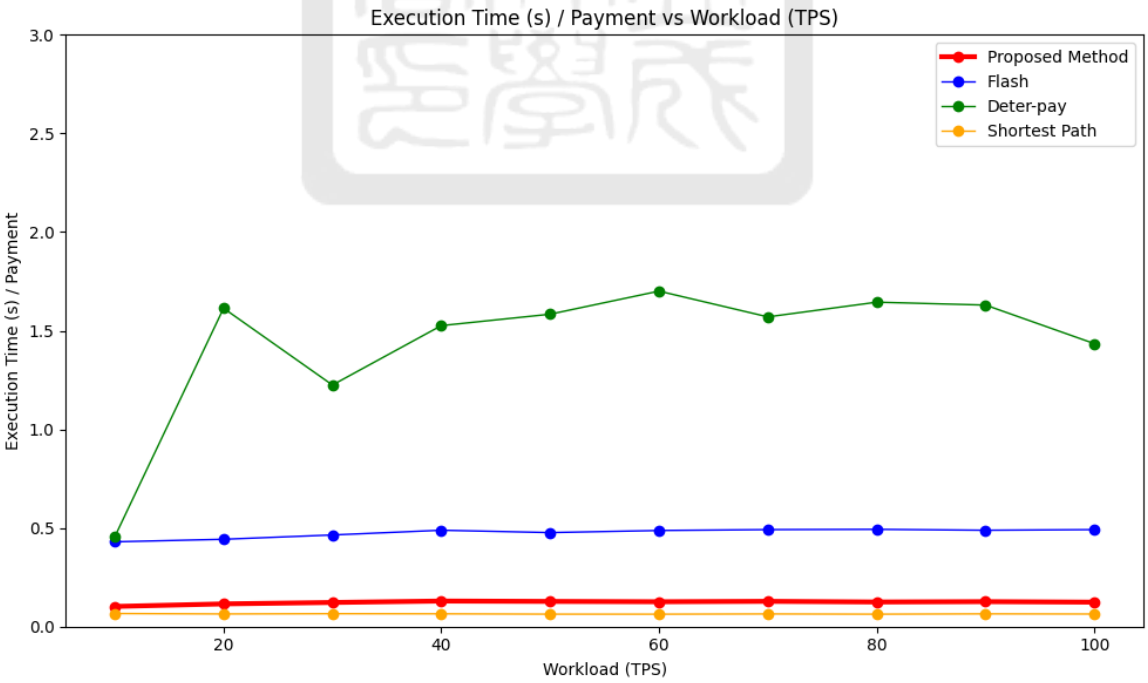

Figure 5.2: Execution time under different workloads.

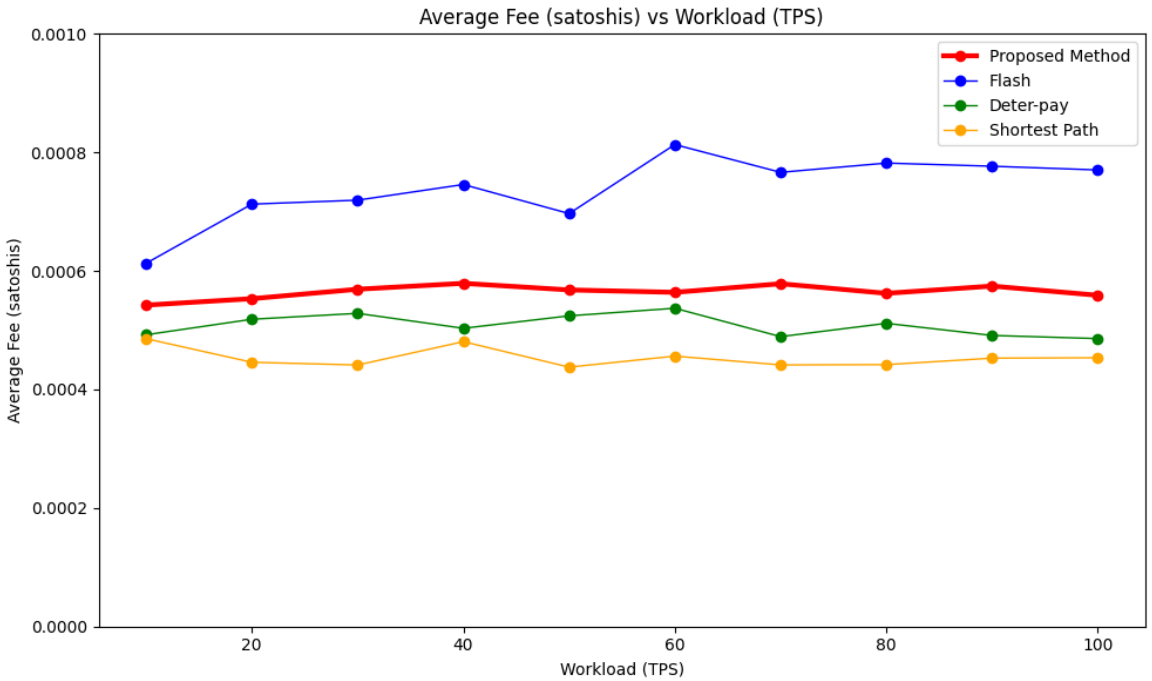

Figure 5.3: Transaction fees under different workloads.

loads. The method achieves the highest success rate even at a transaction frequency of 100 TPS and demonstrates its efficiency under high-load conditions. In contrast, Deter-Pay and Flash exhibit fluctuating or declining success rates as the workload increases, and Shortest Path consistently has the lowest performance among all evaluated methods.

Transaction Latency: Figure 5.2 displays the variation in transaction delay with different transaction workload frequencies. Compared to Deter-Pay and Flash, the proposed method exhibits stable and comparatively lower execution times. Shortest Path that depends solely on the most basic shortest path algorithm based on network topology, achieves the highest execution efficiency. The proposed method employs a two-phase payment process. A successful first payment phase can reduce overall transaction delay. Deter-Pay's requirement for path probing and channel reservation contributes to its higher transaction delay, especially as higher transaction frequen-

cies may necessitate multiple probing and reservation attempts. Flash exhibits lower latency than Deter-Pay, but the method may incur higher delays due to repeated attempts. Flash lacks effective adaptation because of prior failures.

Transaction Fees: Figure 5.3 illustrates the variation in transaction fees under different transaction workloads. The proposed method incurs slightly higher fees than Deter-Pay but demonstrates a significant fee advantage over Flash. Shortest Path consistently yields the lowest transaction fees. The result is attributed to its reliance on a single path and a short hop count.

5.3 Performance with Scaled Capacities

Currently, the average channel capacity of the Lightning Network is approximately 900,000 Satoshis. As off-chain payments become more widespread and more PCNs are practically deployed, channel capacities are expected to increase. This experiment was conducted to verify the effectiveness of the proposed method in the future development of payment channels. The simulation was examined with channel capacities scaled from one-fold to tenfold. These simulations were performed with a fixed testing duration and a constant transaction frequency.

Figure 5.4 illustrates that the transaction success rates of all routing methods increase with the enhancement of channel capacity. Under conditions of increased channel capacity, the proposed method demonstrates a significant lead over Deter-Pay and Shortest Path methods.

Figure 5.5 indicates a gradual reduction in the execution time of the proposed method as channel capacity increases. This reduction is attributed to the increased likelihood of successful transactions in the algorithm's first payment phase with the

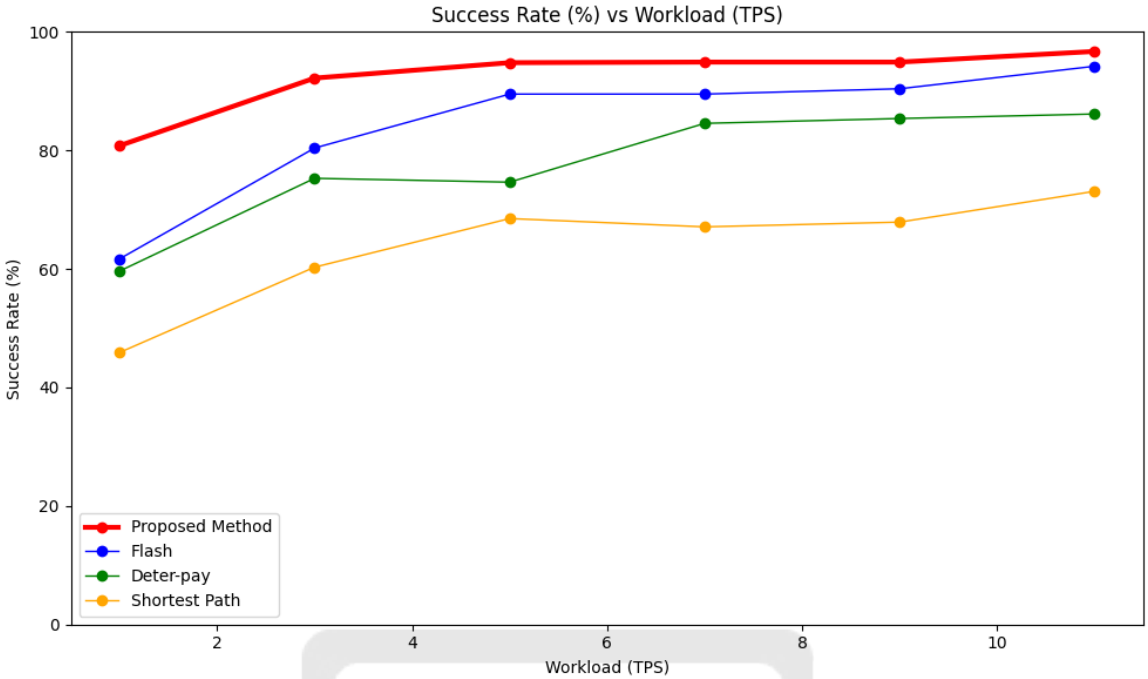

Figure 5.4: Success rate under different channel sizes.

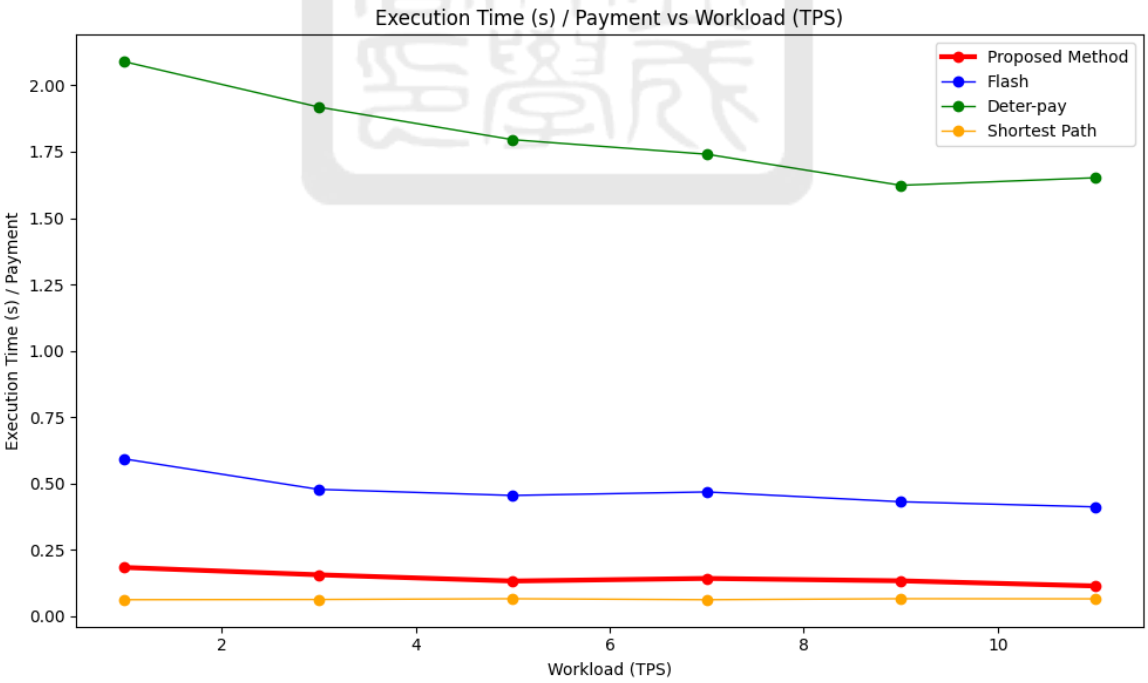

Figure 5.5: Execution time under different channel sizes.

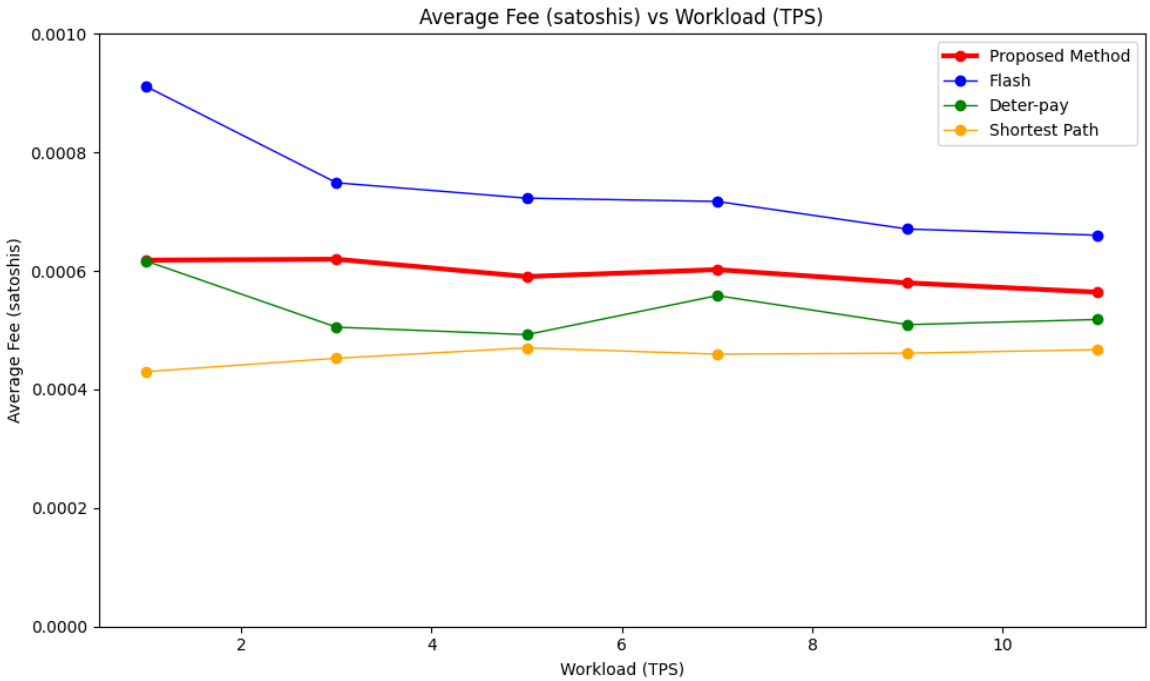

Figure 5.6: Transaction fees under different channel sizes.

larger channel capacity. Path score calculation and transaction splitting are used to achieve this success. This approach reduces the need for second-stage failed handling and consequently lowers transaction delay. In terms of transaction delay, the algorithm exhibits a more significant advantage compared to Deter-Pay and Flash.

Finally, regarding transaction fees, Figure 5.6 shows that the proposed method offers a lower fee advantage compared to Flash. Although its fees are higher than those of Deter-Pay and Shortest Path, the method demonstrates a significant advantage in success rate. The algorithm effectively balances success rate, execution time, and transaction fees.

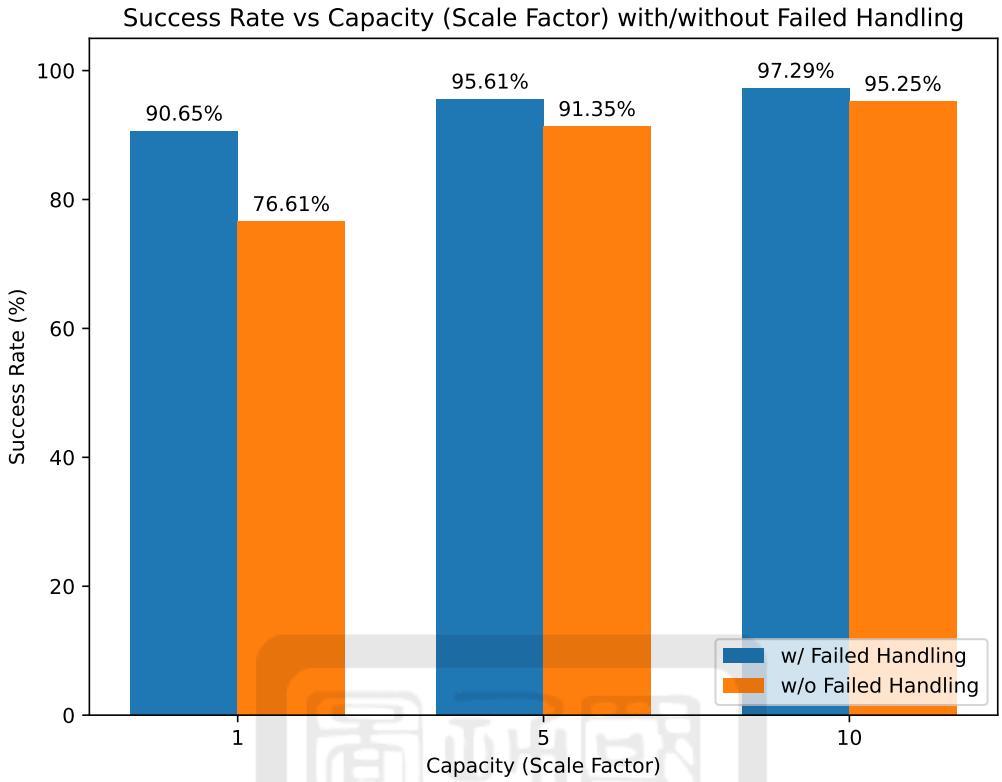

Figure 5.7: Success rate with and without failed handling mechanisms.

5.4 Performance with Failed Handling Mechanisms

The experiment was designed to verify the effectiveness of the algorithm's second phase, called Phase II: Failed Handling. The experiments with or without Phase II were conducted. The results were subsequently tested under different Capacity Scales.

Figure 5.7 reveals that under smaller channel capacity scale factors, the Phase II: Failed Handling mechanism significantly enhances the transaction success rate. When channel capacity is larger, the improvement in success rate by failed handling becomes less prominent. Most transactions can be completed in the first phase. This experimental result corroborates the findings in Section 5.3 regarding lower transaction delay with larger channel capacity. This outcome is similarly attributed to the increased success rate in the first phase.

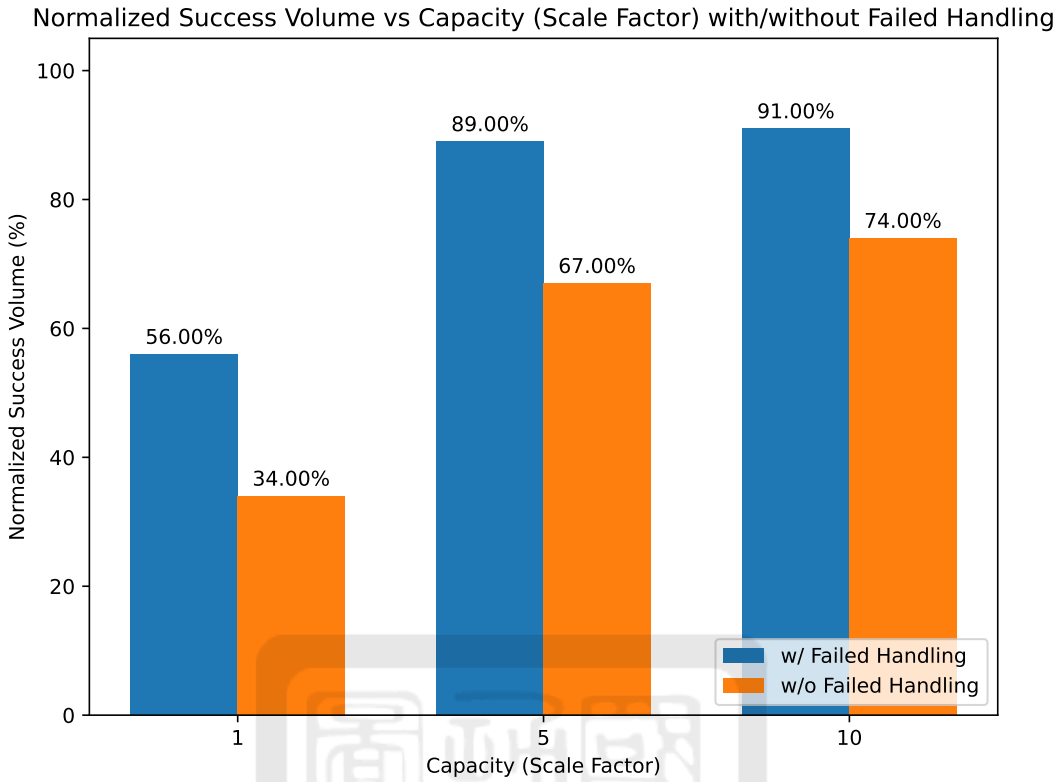

Figure 5.8: Successful volume with and without failed handling mechanisms.

Figure 5.8 demonstrates a significant increase in the total successful transaction volume ratio regardless of the channel capacity. The Phase II failed handling method effectively satisfies large transaction demands and enhances transaction completion rates. The results are achieved by enabling re-attempts based on probing results and intelligently allocating large transaction amounts to available channels.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis presents a two-phase routing algorithm to improve efficiency and reliability in PCNs. The algorithm consists of an initial payment attempt and a re-exploration phase triggered upon failure. Adaptive responses can be enabled to dynamic or inconsistent network conditions.

Routing decisions are informed by a combination of factors, including channel capacity, balance status, fee conditions, and usage patterns. The integrated approach improves path selection, reduces payment failure rates, and lowers overall transaction costs.

Routing performance was examined based on the simulated transaction workloads and network conditions. The proposed algorithm typically outperforms existing methods in terms of success rate, latency, and cost. The evaluation results confirm the effectiveness of the algorithm for scalable PCN routing.

6.2 Future Work

In this research, it is assumed that channel-related information can be obtained through trusted nodes. Nevertheless, considering the high demands for privacy protection in decentralized platforms, it is imperative that no sensitive data is compromised through any means. Therefore, future work ought to explore the development of more efficient routing mechanisms and also guarantee the privacy of every node.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Bitcoin White Paper*, Oct. 2008, Accessed: May 26, 2025. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] BitInfoCharts, "Bitinfocharts.com: Bitcoin statistics," May 2025, Accessed: May 13, 2025. [Online]. Available: https://bitinfocharts.com/bitcoin/

[3] Q. Zhou, H. Huang, Z. Zheng, and J. Bian, "Solutions to scalability of blockchain: A survey," *IEEE Access*, vol. 8, pp. 16 440–16 455, Jan. 2020.

[4] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," Jan. 2016, white Paper. [Online]. Available: https://lightning.network/lightning-network-paper.pdf

[5] Raiden Network, "Raiden network," May 2025, official Website. [Online]. Available: https://raiden.network/

[6] M. Dong, J. Liu, X. Li, and Q. Liang, "Celer network: Bring internet scale to every blockchain," Sept. 2018, Accessed: May 26, 2025. [Online]. Available: https://celer.network/doc/CelerNetwork-Whitepaper.pdf

[7] Bitcoin Wiki Contributors, "Multi-signature," Apr. 2024, Accessed: May 26, 2025. [Online]. Available: https://en.bitcoin.it/wiki/Multi-signature

[8] Bitcoin Wiki Contributors (HTLC), "Hash time locked contracts," Apr. 2024, Accessed: May 26, 2025. [Online]. Available: https://en.bitcoin.it/wiki/Hash_Time_Locked_Contracts

[9] Q. Cai, J. Chen, D. Luo, G. Sun, H. Yu, and M. Guizani, "Deter-pay: A deterministic routing protocol in concurrent payment channel network," *IEEE Internet of Things Journal*, vol. 11, no. 19, pp. 31 206–31 220, Oct. 2024.

[10] R. Yu, G. Xue, V. T. Kilari, D. Yang, and J. Tang, "Coinexpress: A fast payment routing mechanism in blockchain-based payment channel networks," *Proceedings of the 27th International Conference on Computer Communication and Networks*, pp. 1–9, July 2018.

[11] P. Wang, H. Xu, X. Jin, and T. Wang, "Flash: Efficient dynamic routing for offchain networks," *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies*, pp. 370–381, Dec. 2019.

[12] H. Xue, Q. Huang, and Y. Bao, "Epa-route: Routing payment channel network with high success rate and low payment fees," *Proceedings of the 41st International Conference on Distributed Computing Systems*, pp. 227–237, July 2021.

[13] X. Wang, R. Yu, D. Yang, G. Xue, H. Gu, Z. Li, and F. Zhou, "Fence: Fee-based online balance-aware routing in payment channel networks," *IEEE/ACM Transactions on Networking*, vol. 32, no. 2, pp. 1661–1676, Oct. 2023.

[14] Y. Chen, Y. Ran, J. Zhou, J. Zhang, and X. Gong, "Mpcn-rp: A routing protocol for blockchain-based multi-charge payment channel networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1229–1242, Dec. 2021.

[15] B. Ladóczki and Z. Luo, "Routing fee adjustment strategies in payment channels," *Proceedings of the 5th International Conference on Blockchain Computing and Applications*, pp. 7–11, Dec. 2023.

[16] A. Loquercio, A. Saviolo, and D. Scaramuzza, "Autotune: Controller tuning for high-speed flight," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4432–4439, Jan. 2022.

[17] M. Xie and X. He, "Paco: Efficient routing in payment channel networks," *Proceedings of the 3rd International Conference on Computer Science and Blockchain*, pp. 26–31, Jan. 2023.

[18] Q. Gong, C. Zhou, L. Qi, J. Li, J. Zhang, and J. Xu, "Vein: High scalability routing algorithm for blockchain-based payment channel networks," *Proceedings of the 20th International Conference on Trust, Security and Privacy in Computing and Communications*, pp. 43–50, Oct. 2021.

[19] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, Oct. 1999.

[20] Bitcoin Visuals, "Bitcoin visuals - extensive charts and statistics," June 2025, Accessed: June 4, 2025. [Online]. Available: https://bitcoinvisuals.com/

[21] 1ML, "Lightning network search and analysis engine - bitcoin mainnet," June 2025, Accessed: June 4, 2025. [Online]. Available: https://1ml.com/

[22] Machine Learning Group - ULB and Worldline, "Credit card fraud detection dataset," Sept. 2019, Accessed: June 4, 2025. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud